

PROGRAMAÇÃO MODULAR

CONSTRUTORES (E DESTRUTORES)

PROF. JOÃO CARAM

PUC MINAS

BACHARELADO EM ENGENHARIA DE SOFTWARE

CLASSES, OBJETOS E ESTADOS

2

- Considere nossa classe

Pizza:

- Representa uma pizza com preço base fixo;
- Permite operações de modificar os ingredientes.

Pizza
- _maxIngredientes: int - _precoBase: double - _quantidadeIngredientes: int - _valorPorAdicional: double
+ Pizza() - ValorAdicionais() : double - PodeAlterarIngredientes(quantos: int) : bool + ValorFinal() : double + AdicionarIngredientes(quantos: int) : int + RetirarIngredientes(quantos: int) : int + NotaDeCompra() : string

CLASSES E OBJETOS

```
//objeto da Classe Pizza  
Pizza pizza1 = new Pizza();
```



CLASSES E OBJETOS

```
//objeto da Classe Pizza
```

```
Pizza pizza1 = new Pizza();
```

```
IO.println(pizza1.notaDeCompra());
```

CLASSES E OBJETOS

//objeto da Classe Pizza

Pizza pizza1 = new Pizza();

IO.println(pizza1.notaDeCompra());

Quanto vale a pizza neste momento?
Quantos ingredientes tem?

CLASSES E OBJETOS

6

```
//objeto da Classe Pizza  
Pizza pizza1 = new Pizza();
```

- O que acontece com o objeto em sua criação?

Quanto valem
estes atributos?

Pizza
- _maxIngredientes: int - _precoBase: double - _quantidadeIngredientes: int - _valorPorAdicional: double
+ Pizza() - ValorAdicionais() : double - PodeAlterarIngredientes(quantos: int) : bool + ValorFinal() : double + AdicionarIngredientes(quantos: int) : int + RetirarIngredientes(quantos: int) : int + NotaDeCompra() : string

CONSTRUTORES

7

- Métodos construtores são usados para criar e iniciar objetos com valores válidos diferentes do padrão:
 - Em geral, possuem o mesmo nome da classe.
 - Não possuem valores de retorno.
- Uma classe pode ter de um (oculto) a muitos construtores.

BOAS RAZÕES PARA CRIAR CONSTRUTORES

8

- Estado inicial padrão pode não ser aceitável;
- Fornecer um estado inicial consistente é conveniente ao criar objetos;
- Construir um objeto aos poucos é desgastante;
- Um construtor pode ser privado, restringindo assim o uso de sua classe.

CONSTRUTOR PADRÃO (*DEFAULT*)

9

- Utilizados pela linguagem quando não há método construtor implementado.
- Geralmente inicializa todos os atributos com *null*, *false* ou zero.

```
Pizza pizza1 = new Pizza();
```

CONSTRUTOR PADRÃO (*DEFAULT*)

10

- Utilizados pela linguagem quando não há método construtor implementado.
- Geralmente inicializa todos os atributos com *null*, *false* ou zero.
- Mas... Pode simplesmente não inicializar nada.

CONSTRUTOR PADRÃO (*DEFAULT*)

11

- ▣ O construtor padrão pode ser inseguro ou levar a estados inválidos de objetos.

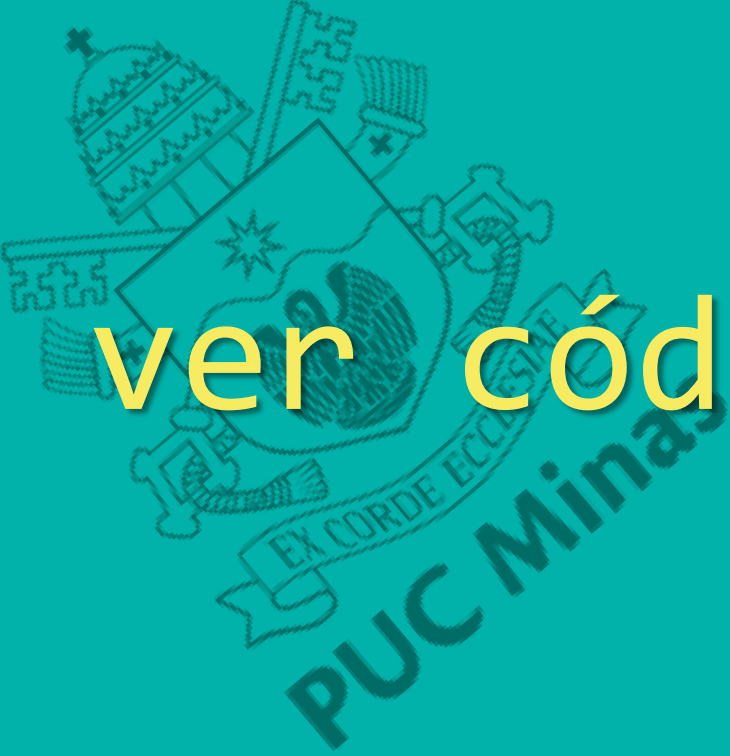
```
Pizza pizza1 = new Pizza();  
IO.println(pizza1.notaDeCompra());
```

**Temos uma pizza válida?
Ela deveria ser válida?**

CONSTRUTOR PRÓPRIO

12

- Implementa-se o construtor próprio para se ter certeza de que as regras definidas para a classe estão sendo seguidas.

A large, faint watermark of the PUC Minas logo is centered in the background. It features a shield with a star, a banner with the motto "EX CORDE ECCLESIAE", and the text "PUC Minas" below it.

{ Quero ver código!! }

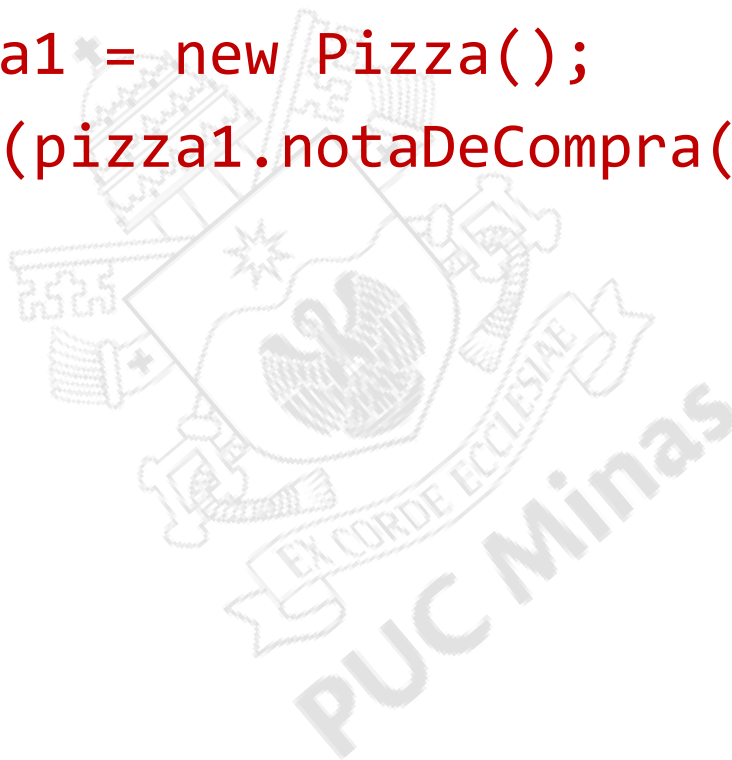
CONSTRUTOR PRÓPRIO

```
public class Pizza {  
    private int maxIngredientes;  
    private double precoBase;  
    private double valorPorAdicional;  
    private int quantidadeIngredientes;  
    public Pizza() {  
        maxIngredientes = 8;  
        precoBase = 29d;  
        valorPorAdicional = 5d;  
        quantidadeIngredientes = 0;  
    }  
}
```

CONSTRUTOR PRÓPRIO

No app:

```
Pizza pizza1 = new Pizza();  
IO.println(pizza1.notaDeCompra());
```



CONSTRUTORES

16

- Procure criar seus construtores próprios.
 - Inicialização correta e completa.
- Na maioria das linguagens OO, uma classe pode ter vários construtores, variando-se a quantidade ou tipos de parâmetros.

MÚLTIPLOS CONSTRUTORES

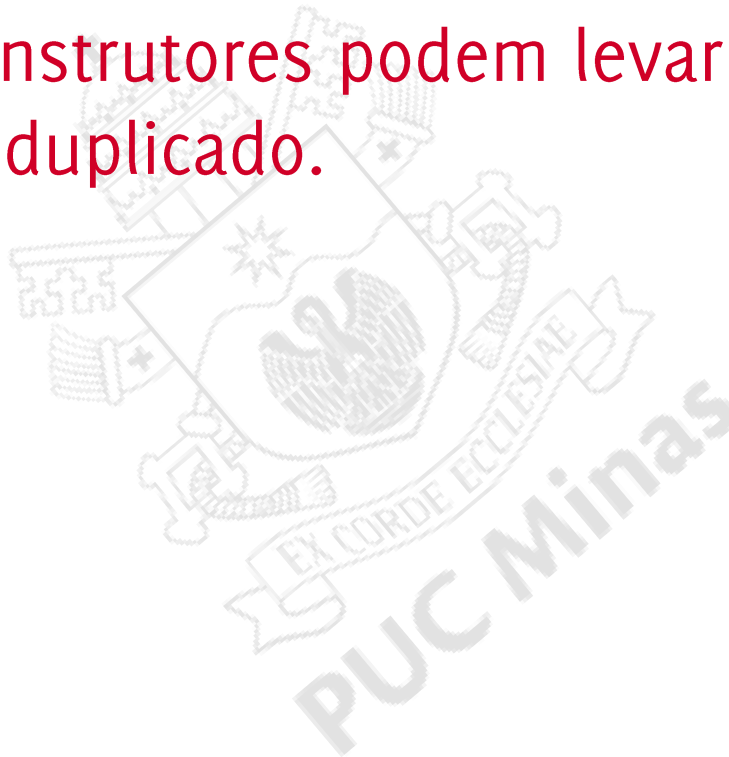
```
public Pizza() {  
    maxIngredientes = 8;  
    precoBase = 29d;  
    valorPorAdicional = 5d;  
    quantidadeIngredientes = 0;  
}
```

```
public Pizza(int adicionais){  
    maxIngredientes = 8;  
    precoBase = 29d;  
    valorPorAdicional = 5d;  
    adicionarIngredientes(adicionais);  
}
```

MÚLTIPLOS CONSTRUTORES

18

- Múltiplos construtores podem levar a inconsistências ou a código duplicado.



MÚLTIPLOS CONSTRUTORES

```
public Pizza() {  
    maxIngredientes = 8;  
    precoBase = 29d;  
    valorPorAdicional = 5d;  
    quantidadeIngredientes = 0;  
}
```

```
public Pizza(int adicionais){  
    maxIngredientes = 8;  
    precoBase = 29d;  
    valorPorAdicional = 5d;  
    adicionarIngredientes(adicionais);  
}
```

MÚLTIPLOS CONSTRUTORES

```
public Pizza() {  
    maxIngredientes = 8;  
    precoBase = 29d;  
    valorPorAdicional = 5d;  
    quantidadeIngredientes = 0;  
}
```

Código duplicado!!

```
public Pizza(int adicionais){  
    maxIngredientes = 8;  
    precoBase = 29d;  
    valorPorAdicional = 5d;  
    adicionarIngredientes(adicionais);  
}
```

MÉTODO INICIALIZADOR

21

- Um artifício comum é criar um método inicializador, o qual pode ser chamado pelos diferentes construtores.

MÉTODO INICIALIZADOR

```
private void init(int adicionais){  
    maxIngredientes = 8;  
    precoBase = 29d;  
    valorPorAdicional = 5d;  
    adicionarIngredientes(adicionais);  
}
```

INICIALIZADOR E CONSTRUTORES

//O inicializador é utilizado pelos construtores

```
public Pizza(){  
    ➡ init(0);  
}
```

```
public Pizza(int adicionais){  
    ➡ init(adicionais);  
}
```

INICIALIZADOR

24

- ▣ Outra vantagem: Estado oculto!
- ▣ Mudanças na regra de inicialização têm impacto apenas no método inicializador.

OBRIGADO.

DÚVIDAS?

26

BÔNUS: DESTRUTORES



DESTRUTORES (*DESTRUCTORS*)

27

- Métodos especiais invocados quando um objeto é finalizado (capturado pelo coletor de lixo).
 - Só há um destrutor por classe.
 - Um destrutor não tem parâmetros.
 - Não deve ser chamado diretamente.

DESTRUTORES (*DESTRUCTORS*)

28

- Objetivo: Liberação de recursos usados pelo objeto
- Ex: memória
arquivos
conexões de rede, bancos de dados..

DESTRUTORES (C++)

```
~nomeDaClasse(){  
    //seu código aqui  
}
```

COLETOR DE LIXO

30

- Processo que libera automaticamente memória que não está sendo mais utilizada.
- Eliminam a necessidade de se desalocar memória explicitamente;
- Eliminam o vazamento de memória;
- Eliminam referências pendentes (*dangling pointer*).

JAVA E COLETOR DE LIXO

31

- ▣ Linguagem Java:
 - ▣ não permite acesso direto à memória;
 - ▣ Não possui operadores de liberação de memória;
 - C, C++: *free, delete*
 - ▣ Possui coletor de lixo (garbage collector).

JAVA E COLETOR DE LIXO

32

- Um objeto é elegível para coleta de lixo quando:
 - não é mais acessado por nenhuma referência;
 - referencia um outro objeto que também o referencia, formando um ciclo único e isolado.

DESTRUTORES JAVA E COLETOR DE LIXO

33

- ▣ O coletor de lixo é autônomo.
 - ▣ Método `System.gc()`.
- ▣ Em Java, os destrutores são chamados automaticamente pelo coletor de lixo.
 - ▣ Execução do método `finalize()` da classe.
 - ▣ `Finalize`: Método obsoleto (`deprecated`).

DESTRUTORES (JAVA)

34

▣ Java 9: Classe *Cleaner* e método *clean()*;

▣ Class Cleaner

<https://docs.oracle.com/en%2Fjava%2Fjavase%2F22%2Fdocs%2Fapi%2F%2F/java.base/java/lang/ref/Cleaner.html>

▣ Java Cleaners: The Modern Way to Manage External Resources

<https://dev.to/ahmedjaad/java-cleaners-the-modern-way-to-manage-external-resources-4d4>

C# E DESTRUTORES

35

- Há uma classe estática GC em C#
 - *Chamada manual da coleta.*
- Classes em C# podem implementar a interface *IDisposable*.
 - Método *Dispose()*.
 - Liberação de recursos sem destruir o objeto.